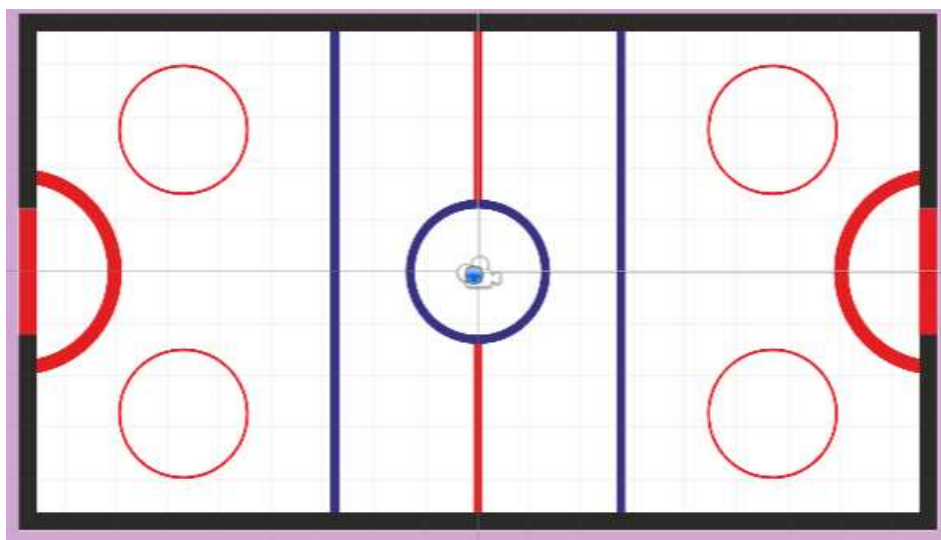


Relatório do trabalho de grupo

AirHockey



Bruno Flamengo a13225

Andreia Mendes a13212

Pedro Barreiro a12313

Programação de jogos em rede, EDJD.

Índice

Introdução.....	3
Classes	4
StartGame.cs	4
Score.cs	5
PlayerMovement.cs	6
DiskMovement.cs.....	8
TCPClientManager.cs	9
ServerManager.cs	11
Conclusão.....	13

Introdução

Neste trabalho foi-nos pedido a realização de um jogo em rede com recurso ao namespace System.net em C#. Optámos por criar um jogo de Air Hockey, onde 2 jogadores controlam um disco cada um num campo de jogo retangular, com o objetivo de inserir um disco neutro, mais pequeno, na baliza do oponente.

Classes

[StartGame.cs](#)

A classe responsável pelo início do jogo.

```
public class StartGame : MonoBehaviour

{
    public GameObject Score;
    public GameObject Disk;
    public bool start = true;

    1 reference
    public void PlayBall()
    {
        if (start)
        {
            Score.SetActive(true);
            Instantiate(Disk);
            start = false;
        }
    }
}
```

Score.cs

A classe responsável por gerir a pontuação dos jogadores.

```
public class Score : MonoBehaviour {

    float player1score, player2score;

    public float winScore;
    public Text speedCount, player1, player2;

    public bool winCondition = false;

    1 reference
    public void ScoreBoard(string player)
    {
        if (player == "Player1Goal")
        {
            player2score++;
            player2.text = player2score.ToString();
        }
        else if (player == "Player2Goal")
        {
            player1score++;
            player1.text = player1score.ToString();
        }

        if (player1score == winScore || player2score == winScore)
            winCondition = true;
    }

    1 reference
    public void SpeedCounter(float speed)
    {
        speedCount.text = speed.ToString();
    }
}
```

[PlayerMovement.cs](#)

Classe que controla o movimento dos jogadores.

```
Transform player;
Vector3 position, oldPosition;
float x, y;

public float speed, radius, limitMin, limitMax;
public bool Playable;
public Vector3 movement;

[HideInInspector]
public TCPClientManager clientManager;

0 references
void Start () {
    player = GetComponent<Transform>();
    position = player.position;
}

0 references
void Update () {
    if (!Playable) return;

    x = Input.GetAxis("Horizontal");
    y = Input.GetAxis("Vertical");

    movement = new Vector3(x, y, 0);
    movement.Normalize();

    if (player.position.x - radius < limitMin)
        player.position = new Vector3(limitMin + radius, player.position.y, 0);
    else if (player.position.x + radius > limitMax)
        player.position = new Vector3(limitMax - radius, player.position.y, 0);
}
```

```
void FixedUpdate()
{
    if (!Playable) return;

    player.Translate(movement * speed * Time.deltaTime);

    if (player.position != oldPosition)
    {
        Message msg = new Message();
        msg.MessageType = MessageType.PlayerMovement;
        PlayerInfo info = new PlayerInfo();
        info.Id = clientManager.Player.Id;
        info.X = transform.position.x;
        info.Y = transform.position.y;
        msg.PlayerInfo = info;
        clientManager.SendMessage(msg);
    }

    oldPosition = transform.position;
}

1 reference
public void Return()
{
    player.position = position;
}
```

DiskMovement.cs

Classe que controla o movimento do disco neutro.

```
void Start () {
    disk = GetComponent<Transform>();
    originalSpeed = speed;
    LaunchDisk();
    players = GameObject.FindGameObjectsWithTag("Player");
    score = GameObject.FindGameObjectWithTag("Score").GetComponent<Score>();
}

0 references
void Update () {
    score.SpeedCounter(speed);
    disk.Translate(direction * speed * Time.deltaTime);

    if (score.winCondition == true)
    {
        gameObject.SetActive(false);
        foreach (GameObject p in players)
            p.SetActive(false);
    }
}
```

```
void LaunchDisk()
{
    disk.position = Vector3.zero;
    speed = originalSpeed;

    do
    {
        direction = new Vector3(Random.Range(-1, 2), Random.Range(-1, 2), 0);
    }
    while (direction == new Vector3(0, 0, 0) || direction.x == 0);
}

0 references
void OnCollisionEnter2D(Collision2D collision)
{
    switch (collision.gameObject.tag)
    {
        case "Player":
            Vector3 player = collision.gameObject.GetComponent<PlayerMovement>().movement;
            //speed++;
            if (player == Vector3.zero)
                direction = -direction;
            else direction = player;
            break;

        case "Wall":
            if (collision.gameObject.name == "WallUp" || collision.gameObject.name == "WallDown")
                direction = Vector3.Reflect(direction, Vector3.up);
            else if (collision.gameObject.name == "WallLeft" || collision.gameObject.name == "WallRight")
                direction = Vector3.Reflect(direction, Vector3.right);
            break;

        case "Finish":
            score.ScoreBoard(collision.gameObject.name);
            LaunchDisk();
            foreach (GameObject p in players)
                p.GetComponent<PlayerMovement>().Return();
            break;
    }
}
```


TCPClientManager.cs

Classe que recebe informação dos jogadores e dirige-a ao servidor.

```
private void Awake()
{
    Player = new Player();
    _playerGameObjectDict = new Dictionary<Guid, GameObject>();
    Player.GameState = GameState.Disconnected;
    Player.TcpClient = new TcpClient();
}

0 references
void Update()
{
    if (Player.TcpClient.Connected)
    {
        switch (Player.GameState)
        {
            case GameState.Connecting:
                Debug.Log("Connecting");
                Connecting();
                break;
            case GameState.Connected:
                Debug.Log("Connected");
                Connected();
                break;
            case GameState.Sync:
                Debug.Log("Syncing");
                Sync();
                break;
            case GameState.GameStarted:
                Debug.Log("GameStarted");
                GameStarted();
                break;
        }
    }
}
```

```
private void Connecting()
{
    if (Player.TcpClient.GetStream().DataAvailable)
    {
        string playerJsonString = Player.BinaryReader.ReadString();
        Player player = JsonConvert.DeserializeObject<Player>(playerJsonString);
        Player.Id = player.Id;
        Player.MessageList.Add(player.MessageList.FirstOrDefault());

        string newPlayerJsonString = JsonConvert.SerializeObject(Player);
        Player.BinaryWriter.Write(newPlayerJsonString);
        Player.GameState = GameState.Connected;
    }
}
```

1 reference

```
private void Connected()
{
    if (Player.DataAvailable())
    {
        Message message = ReceiveMessage();
        Debug.Log(message.Description);
        Player.GameState = GameState.Sync;
    }
}
```

```
private void AcceptConnection(IAsyncResult ar)
{
    TcpClient tcpClient = (TcpClient)ar.AsyncState;
    tcpClient.EndConnect(ar);

    if (tcpClient.Connected)
    {
        Debug.Log("Client connected");
        Player.BinaryReader = new System.IO.BinaryReader(tcpClient.GetStream());
        Player.BinaryWriter = new System.IO.BinaryWriter(tcpClient.GetStream());
        Player.MessageList = new List<Message>();
    }
    else
    {
        Debug.Log("Client connection refused");
    }
}
```

[ServerManager.cs](#)

Classe que contém o servidor do nosso jogo.

```
foreach (Player player in players)
{
    switch (player.GameState)
    {
        case GameState.Connecting:
            if (player.DataAvailable())
            {
                Console.WriteLine("New player registering");
                string playerJson = player.BinaryReader.ReadString();
                Player playerMsg = JsonConvert.DeserializeObject<Player>(playerJson);

                foreach (Player notifyPlayer in players)
                {
                    Message msg = new Message();
                    msg.MessageType = MessageType.NewPlayer;
                    msg.Description = (notifyPlayer == player) ?
                        "Successfully joined" :
                        "Player has joined";
                    PlayerInfo playerInfo = new PlayerInfo();
                    playerInfo.Id = player.Id;
                    playerInfo.X = 0;
                    playerInfo.Y = 0;
                    msg.PlayerInfo = playerInfo;

                    string msgJson = JsonConvert.SerializeObject(msg);
                    notifyPlayer.BinaryWriter.Write(msgJson);
                    notifyPlayer.MessageList.Add(msg);
                    Console.WriteLine(msgJson);
                }
                player.GameState = GameState.Sync;
            }
    }
}

private void SyncPlayerMovement(Player player)
{
    foreach (Player p in players)
    {
        if (p.GameState == GameState.GameStarted)
        {
            var last = p.MessageList.LastOrDefault(
                m => m.MessageType == MessageType.PlayerMovement);
            if (last != null)
            {
                Message msg = new Message();
                msg.PlayerInfo = last.PlayerInfo;
                string jsonMsg = JsonConvert.SerializeObject(msg);
                player.BinaryWriter.Write(jsonMsg);
            }
        }
    }
}
```

```
private void AcceptTcpClient(IAsyncResult ar)
{
    TcpListener tcpListener = (TcpListener)ar.AsyncState;
    TcpClient tcpClient = tcpListener.EndAcceptTcpClient(ar);
    if (tcpClient.Connected && players.Count < numPlayers)
    {
        Console.WriteLine("Accepted new connection");

        Player player = new Player();
        Message message = new Message();
        message.Description = "Hello new player";
        message.MessageType = MessageType.Information;

        player.MessageList = new List<Message>();
        player.MessageList.Add(message);
        player.TcpClient = tcpClient;
        player.BinaryReader = new System.IO.BinaryReader(tcpClient.GetStream());
        player.BinaryWriter = new System.IO.BinaryWriter(tcpClient.GetStream());
        player.Id = Guid.NewGuid();
        player.GameState = GameState.Connecting;

        players.Add(player);

        string playerJson = JsonConvert.SerializeObject(player);
        Console.WriteLine(playerJson);
        player.BinaryWriter.Write(playerJson);
    }
    else
    {
        Console.WriteLine("Connection refused");
    }
}

public void StartServer()
{
    TcpListener listener = new TcpListener(IPAddress.Parse("192.168.1.73"), 7777);
    listener.Start();
    Console.WriteLine("Server On");
}
```

Conclusão

Com este trabalho aprendemos a realizar um jogo em rede recorrendo a protocolos TCP.